

Engine Management Unit (EMU)

Hardware and Software Reference

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.9

Date: September 2026

This document describes the hardware, software architecture, valve control system, Fill valve handshake protocol, sensor subsystems, buzzer and visual indicator state machine, and control flow of the Engine Management Unit. The EMU is mounted inside the rocket and is the primary actuator and sensor node of the KJO system.

Contents

1	Hardware Overview	2
1.1	Component Summary	2
1.2	MCP23X17 GPIO Expander Pin Map	3
1.3	Hardware Block Diagram	4
2	Software Architecture	4
2.1	Module Summary	4
2.2	Main Loop Control Flow	5
3	Valve Control Subsystem	5
3.1	Valve Base Class (KJO_Valve)	5
3.2	EMU State Machine	5
3.3	Engine Start Sequence (10 Steps)	6
3.4	Servo Mechanical Geometry and Calibration	7
3.5	Local Button Functions	8
4	Fill Valve CAN Protocol	8
5	Command Dispatch	9
5.1	Command Table	10
6	Umbilical Quick-Release (QR_Master)	10
6.1	Overview	11
7	Buzzer and Visual Indicator	11
7.1	Startup Self-Test	11
7.2	NeoPixel Indicator State Machine	11
7.3	Buzzer Behaviour	12
7.4	NeoPixel_Indicator API	12
8	Sensor Subsystems	13
8.1	Pressure Transducer Signal Chain	13
8.2	Continuous Pressure Recording	13
9	SD Logging	14
10	Parameters & Flags	14
11	Version History	20

1 Hardware Overview

1.1 Component Summary

Component	Part	Interface	Notes
Microcontroller	Adafruit Feather M0 (AT-SAMD21G18A, 48 MHz)	—	
OLED Display	SH1107, 128×64, 1.12"	I ² C	
GPIO Expander	MCP23X17 (16 I/O)	I ² C	Relays, buttons, AUX_IO
PWM Servo Wing	Adafruit 8-channel wing (PCA9685)	I ² C	Ox, Fuel servos
ADC	ADS1015 (12-bit, 4-ch)	I ² C	Ch0: unassigned; Ch1: chamber pressure; Ch2: tank pressure; Ch3: battery voltage
Thermocouple IF	MAX31856 (2×)	SPI	TC1, TC2
Real-Time Clock	PCF8523	I ² C	
LoRa Radio	HopeRF RFM95W (Semtech SX1276), 915 MHz	SPI	
SD Card	Standard SPI SD socket	SPI	
NeoPixel	WS2812B 4×8 array	GPIO	Status indicator
Igniter Relay	SPDT relay via MCP23X17	GPIO	
Buzzer	Passive piezo via MCP23X17 relay	GPIO	
Battery	3.7 V LiPo; voltage divider on ADS1015	ADC	

1.2 MCP23X17 GPIO Expander Pin Map

Pin Name	Direction	Function
BUTTON_A_EIO	IN	Front-panel button A (hold-to-open Ox valve, dumps via chamber)
BUTTON_B_EIO	IN	Front-panel button B; currently unused (see §3.5)
BUTTON_C_EIO	IN	Front-panel button C; currently unused (see §3.5)
IGNITER_EIO	OUT	Igniter relay drive
BUZZER_EIO	OUT	Buzzer relay drive
AUX_IO_1 (EMU_LINK_SENSE_EIO)	IN (INPUT_PULLUP)	Umbilical link-disconnect sense; wired to GSEMU chassis GND (LOW = connected, HIGH = separated). Fill valve CMD/STATUS formerly carried here has moved to CAN.
AUX_IO_2	—	Retired/unused (formerly FILL_VALVE_STATUS_EIO).
AUX_IO_3 (QR_SENSE_EIO)	IN (INPUT_PULLUP)	Umbilical QR physical-connection sense; wired to GSEMU chassis GND (LOW = connected, HIGH = separated). QR release itself is commanded over CAN (CAN_QR_RELEASE), not this line.
AUX_IO_4	—	Retired/unused (formerly RELEASE_STATE_EIO; its sense function consolidated onto QR_SENSE_EIO).
AUX_IO_5	—	Retired/unused (formerly LAUNCH_ENABLE_EIO).
AUX_IO_6	—	Retired/unused (formerly REMOTE_START_EIO).

1.3 Hardware Block Diagram

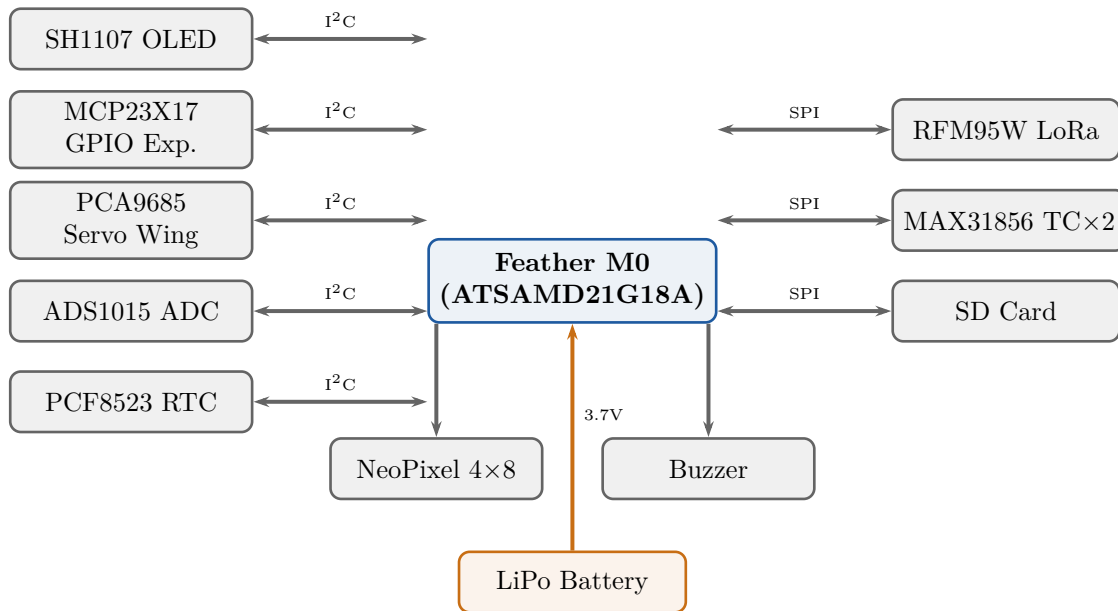


Figure 1: EMU hardware block diagram.

2 Software Architecture

2.1 Module Summary

Module	Location	Responsibility
main.cpp	local	Setup, loop, command dispatch, state machines
KJO_Valve	local	Base valve: servo drive, pot feedback
KJO_QR_Master	local	Umbilical connection-sense reader (see §6)
KJO_Control_System	local	Execute_Command() routing table
KJO_Pressure	local	Pressure transducer read / convert
KJO_Temperature	local	Analog temperature transducer
KJO_Thermocouple	local	MAX31856 thermocouple read
KJO_Analog	local	ADS1015 ADC wrapper
KJO_GPIO	local	MCP23X17 pin definitions, Relay class
KJO_EMU_Display	local	SH1107 OLED display functions
KJO_Indicator	local	NeoPixel status indicator (solid, timed, flash)
KJO_Logging	local	SD log file constants
KJO_Radio	shared	Radio pin constants, frequency
KJO_Command_Defs	shared	Command codes, packet structures
KJO_System_Config	shared	Version, platform constants

2.2 Main Loop Control Flow

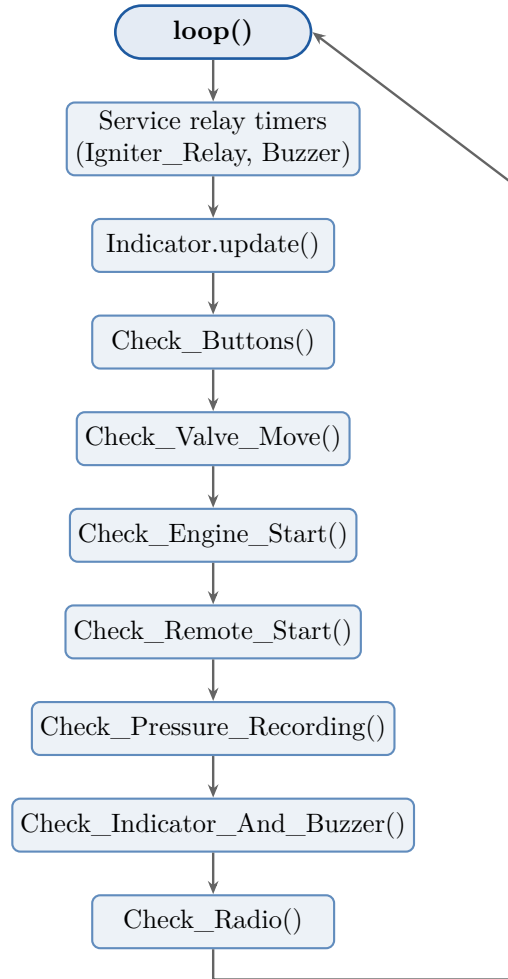


Figure 2: EMU main loop control flow.

3 Valve Control Subsystem

3.1 Valve Base Class (KJO_Valve)

The `Valve` class encapsulates a servo-driven, potentiometer-feedback valve:

- **Drive:** PWM signal to PCA9685 servo wing channel.
- **Feedback:** Potentiometer on ADS1015 ADC channel; position expressed as 0–100% open.
- **Position control:** `openTo(%)` sets target; `update()` drives servo until pot reading reaches target within dead-band.
- **State queries:** `isMoving()`, `isStopped()`, `isOpen()`, `isClosed()`.

3.2 EMU State Machine

The EMU operates as a top-level state machine with three states:

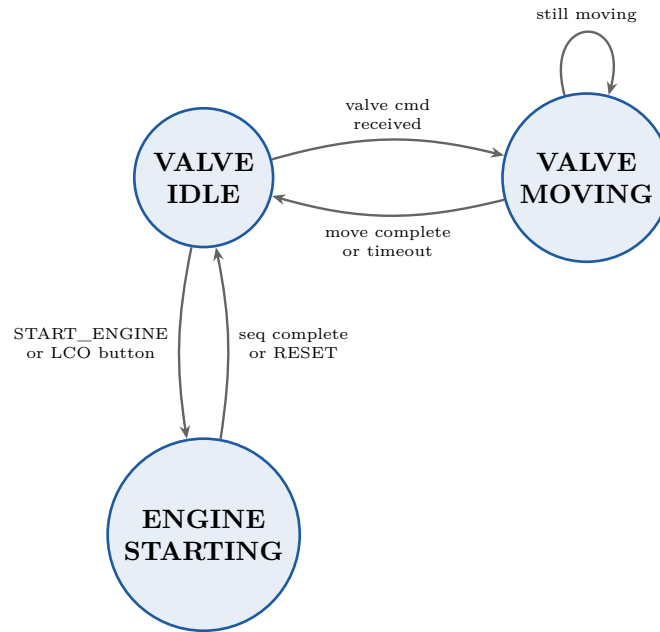


Figure 3: EMU top-level state machine. HEARTBEAT and RESET are processed in any state. All other commands are dropped during VALVE_MOVING or ENGINE_STARTING.

3.3 Engine Start Sequence (10 Steps)

When **START_ENGINE** or a LCO panel button press is received, the EMU enters **ENGINE_STARTING** and advances through ten steps (0–9) non-blockingly in **Check_Engine_Start()**. Propellant flow is established (steps 6–8) *before* the igniter fires (step 9) – reversed from an earlier design that fired the igniter first. Fuel opens before Ox since it has the longer physical path to the chamber and needs the head start.

Step	Trigger	Action
0	immediate	Verify Ox and Fuel valves are closed; abort ($r_val = -1$) on failure
1	step 0 pass	If GSEMU is CAN-reachable, close the Fill valve; otherwise skip to step 3
2	timer / CAN ack	Wait for the Fill valve close to confirm; abort ($r_val = -4$) on failure
3	step 1/2 pass	Send CAN_QR_RELEASE (fire-and-forget); start QR_RELEASE_TIMEOUT_MS separation timer
4	timer	Poll QR_Release.isSeparated() ; abort ($r_val = -2$) on timeout
5	separation	Fixed START_DWELL_MS pause; then open Fuel feed valve
6	Fuel moving	Wait for Fuel valve to confirm open; abort via Execute_Abort() ($r_val = -5$) if stuck/stalled; then start FUEL_FLOW_DELAY_MS timer
7	delay	Open Ox feed valve
8	Ox moving	Wait for Ox valve to confirm open; abort via Execute_Abort() ($r_val = -5$) if stuck/stalled; then start OX_FLOW_DELAY_MS timer
9	delay	Fire igniter relay (IGNITER_FIRE_DURATION_MS pulse); send OK response; set engine_running = true

Any abort at steps 0–4 sends a **Command_Return_Packet** with **status = false** and an **r_val** error code (-1 = precondition failure, -4 = Fill valve close failure, -2 = QR separation timeout). A stuck/stalled Fuel or Ox valve at steps 6 or 8 ($r_val = -5$) routes through **Execute_Abort()** instead, since propellant may already be flowing by that point. **RESET** cancels the sequence at any step.

3.4 Servo Mechanical Geometry and Calibration

Gear Train and Servo Travel

Each valve is actuated by a ServoCity 2000-0025-0504 5-turn servo through a 16-tooth pinion driving a 60-tooth valve gear:

Parameter	Value
Gear ratio	$60 \div 16 = \mathbf{3.75 : 1}$
Valve travel (open $\rightarrow$ close)	90°
Required servo rotation	$90 \times 3.75 = 337.5^\circ$
Servo full mechanical range	1800° (5 turns) over 500–2500 μs
PCA9685 frequency	1100 Hz (non-standard; improves count resolution)
PWM period at 1100 Hz	$1 \div 1100 = 909.09 \mu s$
μs per 12-bit count	$909.09 \div 4096 \approx 0.2219 \mu s/\text{count}$
μs for 337.5° servo travel	$337.5 \times (2000 \mu s \div 1800^\circ) = 375 \mu s$
Theoretical Δ counts	$375 \div 0.2219 \approx \mathbf{1690}$ counts

Note

Running the PCA9685 at 1100 Hz (instead of the standard 50 Hz) compresses the usable pulse-width span into the lower portion of the 12-bit count range, effectively improving resolution over the ~ 1.1 turns of servo travel actually used.

Empirical Calibration (March 2026)

Bench testing showed the theoretical 1690-count span caused all four valves to travel noticeably *more* than 90°. All three valves were re-calibrated. The empirical open-to-close deltas average ≈ 1610 counts ($\sim 5\%$ below theoretical), consistent with the servo being slightly more sensitive in the operating band than the linear model predicts.

Valve	Servo PWM (12-bit counts)				Pot ADC (12-bit counts)		
	Open	Close	Δ	Implied travel	Open	Closed	Δ
Fill	2179	3719	1540	$\sim 82.1^\circ$	1238	2984	1746
Oxidizer	2050	3690	1640	$\sim 87.4^\circ$	1139	2857	1718
Fuel	2130	3780	1650	$\sim 87.9^\circ$	1170	2918	1748

3.5 Local Button Functions

`Check_Buttons()` polls the three front-panel buttons (MCP23X17 expander) every `loop()` iteration, but is skipped entirely while `emu_state` is `ENGINE_STARTING`, `VALVE_MOVING`, `ABORTING`, or `ABORT_LATCHED`.

Button	Action
A	Hold-to-open: opens the Ox valve (dumps pressurant through the chamber) while held; closes it on release. One short beep on press.
B	Currently unused. Formerly commanded the Fill valve closed directly; the Fill valve is now GSEMU's own actuator, commanded only over CAN (<code>BEGIN_FILL/STOP_FILL</code>). Local bench testing of the Fill valve is done via GSEMU's own front-panel buttons.
C	Currently unused, for the same reason as Button B.

Warning

Button A opens the Ox valve immediately while held, venting N_2O through the combustion chamber. Ensure the area is clear before pressing it.

4 Fill Valve CAN Protocol

The Fill valve servo and potentiometer reside on the GSEMU. The wired `AUX_IO_1/AUX_IO_2` CMD/STATUS handshake this section formerly described has been retired; the EMU now controls the Fill valve entirely over CAN.

Open/close sequence:

1. EMU sends `CAN_OPEN_FILL_VALVE` or `CAN_CLOSE_FILL_VALVE` to `CAN_NODE_GSEMU` and awaits a response.

2. GSEMU's `Check_CAN()` calls `Fill_Valve.open()/close()` immediately, but *defers* its CAN response.
3. GSEMU's `Check_Fill_Valve_Move()` (called every `loop()`) sends the deferred response once `Fill_Valve.isStopped()` confirms the move (or its own `MOVE_TIMEOUT`-based deadline expires), reporting the resulting position.
4. EMU treats the request as failed if no response arrives within `EMU_GSEMU_FILL_VALVE_MOVE_TIMEOUT_MS` (see §10 Parameters & Flags), which is sized to cover GSEMU's own move watchdog plus CAN round-trip margin.

5 Command Dispatch

`Check_Radio()` receives packets from the RCU and routes them. Commands are processed in priority order:

1. **HEARTBEAT** — always processed; returns `Heartbeat_Response_Packet` with EMU voltage and all three valve positions.
2. **RESET** — always processed regardless of state; calls `Execute_Abort()`.
3. All other commands are *dropped* if `emu_state` is `VALVE_MOVING` or `ENGINE_STARTING`.
4. Per-command interceptors (deferred valve moves, start sequence, recording).
5. All remaining commands dispatched to `Execute_Command()`.

5.1 Command Table

Command	EMU Action
HEARTBEAT	Respond with EMU voltage and all three valve positions
RESET	Execute_Abort(): Fill/Fuel close, Ox opens (dump), igniter off, buzzer off, recording off, indicator off
BEGIN_FILL	Open Fill valve (CAN to GSEMU); start continuous pressure recording
STOP_FILL	Close Fill valve; recording continues until ambient
DUMP	Open Ox valve to vent pressurant through the chamber (deferred; sends response when complete; no dedicated Dump valve)
STOP_DUMP	Close Ox valve (deferred)
OPEN_OXIDIZER_FEED	Open Ox valve (deferred)
CLOSE_OXIDIZER_FEED	Close Ox valve (deferred)
OPEN_FUEL_FEED	Open Fuel valve (deferred)
CLOSE_FUEL_FEED	Close Fuel valve (deferred)
START_ENGINE_FLOWS	Close Fill; open Ox + Fuel simultaneously (deferred dual-valve)
STOP_ENGINE_FLOWS	Close Ox + Fuel simultaneously (deferred dual-valve)
START_ENGINE	Arm 10-step ignition sequence (response sent asynchronously)
ENABLE_LCO_CONTROL	param $\neq$ 0: assert LAUNCH_ENABLE_EIO LOW, set <code>lco_enabled</code> ; param = 0: deassert HIGH, clear <code>lco_enabled</code>
START_PRESSURE_RECORDING	Activate continuous sensor recording
STOP_PRESSURE_RECORDING	Deactivate continuous sensor recording
READ_VALVE_POSITION	Return current position (%) of the valve specified in <code>param</code>
READ_TANK_PRESSURE	Return tank pressure (PSI)
READ_CHAMBER_PRESSURE	Return chamber pressure (PSI)
READ_TANK_TEMP	Return tank temperature
READ_CHAMBER_TEMP	Return chamber temperature
QR_RELEASE	Bench-test only: reversible timed release via <code>CAN_QR_TEST_RELEASE</code> (GSEMU's local-override mechanism); production launch release happens automatically as Step 3 of the Engine Start sequence via <code>CAN_QR_RELEASE</code>

6 Umbilical Quick-Release (QR_Master)

6.1 Overview

The umbilical QR servo and its drive latch reside on the GSEMU. EMU-side `QR_Master` no longer commands it via a dedicated wire – release is commanded over CAN (`CAN_QR_RELEASE`, fire-and-forget). `QR_Master`'s only remaining job is reading the umbilical's physical connection-sense line:

- `AUX_IO_3 (QR_SENSE_EIO)` — `INPUT_PULLUP`. Wired to GSEMU chassis GND through the AUX connector. Reads LOW while the umbilical is physically connected; floats HIGH when the connector is removed (separation confirmed). Kept as a dedicated wire (not CAN) because once the umbilical separates, EMU and GSEMU can no longer communicate over CAN at all.

`QR_Master` exposes `isConnected()` and `isSeparated()` reads of this line – it has no `release()/hold()` methods. Release is a one-shot CAN command sent directly from `main.cpp`:

- **Production launch:** `CAN_QR_RELEASE` is sent automatically as Step 3 of the Engine Start sequence (see §3.3). GSEMU latches this permanently – once triggered, the spring-loaded latch cannot be electrically re-engaged; only physical re-latching (or GSEMU's own front-panel Button A override) changes it.
- **Bench testing:** `RCU_UNIT_TEST`'s `QR_RELEASE` command instead sends `CAN_QR_TEST_RELEASE`, which drives GSEMU's reversible local-override mechanism (auto-reverting after a fixed duration) and never touches the permanent production latch.

Warning

Ensure the launch sequence has been verified and all personnel are clear before the Engine Start sequence reaches Step 3. The umbilical disconnection releases pressurised N₂O, and production release cannot be electrically undone once commanded.

7 Buzzer and Visual Indicator

7.1 Startup Self-Test

At power-on, the buzzer fires **five short beeps** (75 ms on / 75 ms gap) to confirm the buzzer and relay circuits are functional. The NeoPixel cycles red → yellow → green → off as a pixel self-test.

7.2 NeoPixel Indicator State Machine

`Check_Indicator_And_Buzzer()` runs every loop iteration and updates the 32-pixel NeoPixel array to reflect the highest-priority active condition. States are evaluated in descending priority:

Priority	Condition	Pattern	Colour
1	<code>engine_running = true</code>	Off (all pixels dark)	—
2	<code>emu_state = ENGINE_STARTING</code>	Solid	Red
3	<code>tank_pressurized = true</code>	Flash 300/500 ms	Red
4	<code>link_connected = true</code>	Flash 300/1200 ms	Green
5	(unlinked)	Flash 300/1200 ms	Yellow

`link_connected` is set true on every received **HEARTBEAT** and cleared after 15 s of silence (loss-of-signal watchdog). `tank_pressurized` is updated by `Check_Pressure_Recording()` using the live tank pressure reading; the threshold is `> PRESSURE_WARNING_THRESHOLD` (15 PSI).

7.3 Buzzer Behaviour

Condition	Buzzer Action
Startup self-test	5 beeps (75 ms on, 75 ms gap) — blocking, in <code>setup()</code>
Button press confirmation	1 or 2 short beeps (100 ms) via <code>Beep()</code>
Pressure alarm	Continuous tone while <code>tank_pressurized = true</code>
Engine start / abort	Alarm silenced immediately when <code>Arm_Engine_Start()</code> or <code>Execute_Abort()</code> is called
“I’m alive” heartbeat	Single 100 ms beep every 10 s; suppressed during alarm, <code>ENGINE_STARTING</code> , and <code>ENGINE_RUNNING</code>

The pressure alarm is managed entirely within `Check_Indicator_And_Buzzer()`. When the indicator state transitions *to* `IND_PRESSURIZED`, `Buzzer.on()` is called (no timeout — continuous). When the state transitions *away* from `IND_PRESSURIZED`, `Buzzer.off()` is called. `Arm_Engine_Start()` and `Execute_Abort()` both call `Buzzer.off()` directly and clear the `pressure_alarm_active` flag so the next `Check_Indicator_And_Buzzer()` call does not re-arm it.

7.4 NeoPixel_Indicator API

Method	Action
<code>begin()</code>	Initialise hardware; clear all pixels
<code>on(color)</code>	Set all pixels solid; cancel any flash or timed mode
<code>onRed()</code> / <code>onGreen()</code> / <code>onYellow()</code>	Convenience wrappers
<code>flash(color, on_ms, off_ms)</code>	Periodic flash indefinitely; cancelled by <code>on()</code> or <code>off()</code>
<code>off()</code> / <code>clear()</code>	All pixels off; cancel flash/timed modes
<code>update()</code>	Service timed shutoff and flash state machine — call from <code>loop()</code>

8 Sensor Subsystems

Sensor	Class	ADC / Bus	Measurement
Chamber pressure transducer	Pressure_Transducer	ADS1015 ch1	Combustion chamber pressure, 0–200 PSI
Tank pressure transducer	Pressure_Transducer	ADS1015 ch2	N ₂ O tank pressure, 0–1000 PSI
Battery voltage	—	ADS1015 ch3	LiPo pack voltage (via resistor divider)
Tank temperature sensor	Temperature_Transducer	Onboard ADC (A0)	N ₂ O tank temperature [†]
Chamber temperature sensor	Temperature_Transducer	Onboard ADC (A5)	Chamber wall temperature [†]
Thermocouple 1	MAX31856	SPI	TC1 (location TBD)
Thermocouple 2	MAX31856	SPI	TC2 (location TBD)

[†]Onboard ADC channel assignments are placeholders pending hardware schematic confirmation.

8.1 Pressure Transducer Signal Chain

Each pressure transducer produces a ratiometric voltage output over 0.5–4.5 V corresponding to 0–full-scale PSI. A voltage divider on the A/D daughter board scales this to the ADS1015 input range:

$$V_{\text{ADC}} = V_{\text{transducer}} \times \frac{3.3}{4.5}$$

Condition	Transducer (V)	After divider (V)	ADS1015 count (GAIN_ONE)
0 PSI (zero offset)	0.500	0.367	≈ 184
Full-scale PSI	4.500	3.300	≈ 1657

ADS1015 at GAIN_ONE (± 4.096 V): ≈ 0.001991 V/count. The two-point linear `map()` in `Pressure_Sensor_Average()` implicitly performs all three conversion steps (undo divider, subtract 0.5 V offset, scale 0–4 V to PSI) without explicit voltage arithmetic.

Constant	Value	Meaning
PRESSURE_MIN_READING	184	ADS1015 counts at 0 PSI (0.500 V transducer)
PRESSURE_MAX_READING	1657	ADS1015 counts at full-scale PSI (4.500 V transducer)
CHAMBER_MAX_PSI	200	Chamber transducer full-scale (PSI)
TANK_MAX_PSI	1000	Tank transducer full-scale (PSI)

8.2 Continuous Pressure Recording

Continuous recording is started automatically by `BEGIN_FILL` and may also be started manually with `START_PRESSURE_RECORDING`. Every `PRESSURE_RECORDING_INTERVAL_MS` (250 ms) the EMU reads tank pressure, chamber pressure, tank temperature, and chamber temperature and appends all four values to the SD log.

Recording auto-stops when the tank pressure remains below `PRESSURE_AMBIENT_THRESHOLD` (10 PSI) for four consecutive readings, or when `STOP_PRESSURE_RECORDING` or `RESET` is received.

9 SD Logging

Log files are stored in the root of the SD card as `Log_N.txt` (N is auto-incremented to avoid overwriting existing files). Each record is time-stamped (PCF8523) and records command receipt, valve movements, sensor readings, and fault events.

10 Parameters & Flags

This section collects the tunable operational parameters and feature flags compiled into this firmware image, together with the source file where each is defined. It is meant as a single reference point when tracing or changing a timing, threshold, or calibration value. Pin/channel wiring assignments, LoRa/CAN command opcodes, and packet byte layouts are documented separately (the Hardware Overview pin map and the Command Set sections above) and are not repeated here. Parameters marked (*shared*) are defined in `KJO_Shared_Libraries` and must agree with the same constant used on the other board(s) that consume it.

Valve Servo Timing & Calibration

Parameter	Value	Description	Source File
<code>SERVO_FREQUENCY</code>	1100 Hz	Non-standard PWM frequency (vs. 50 Hz) chosen to compress the usable PWM count range and improve position resolution over the servo's ~1.1-turn travel.	<code>KJO_Valve.h</code>
<code>SERVO_RANGE_MIN</code>	500 μ s	Pulse width at the servo's lower mechanical travel limit.	<code>KJO_Valve.h</code>
<code>SERVO_RANGE_MAX</code>	2500 μ s	Pulse width at the servo's upper mechanical travel limit.	<code>KJO_Valve.h</code>
<code>SERVO_FULL_RANGE_DEGREES</code>	1800°	Total mechanical travel of the 5-turn servo used for position-mapping math.	<code>KJO_Valve.h</code>
<code>SERVO_PINION_TEETH</code>	16	Tooth count of the servo output pinion gear, used in valve gearing geometry.	<code>KJO_Valve.h</code>
<code>VALVE_GEAR_TEETH</code>	60	Tooth count of the valve actuator gear; combined with pinion teeth sets the 60:16 gear ratio.	<code>KJO_Valve.h</code>
<code>VALVE_TURN_DEGREES</code>	90°	Valve rotational travel from full open to full closed.	<code>KJO_Valve.h</code>
<code>OX_VALVE_PWM_OPEN</code>	2200	12-bit PWM count for the Ox valve's fully-open servo position.	<code>KJO_Valve.h</code>
<code>OX_VALVE_PWM_CLOSE</code>	3800	12-bit PWM count for the Ox valve's fully-closed servo position.	<code>KJO_Valve.h</code>
<code>OX_VALVE_POS_OPEN</code>	1141	ADC count from the Ox valve position pot when fully open.	<code>KJO_Valve.h</code>
<code>OX_VALVE_POS_CLOSED</code>	2898	ADC count from the Ox valve position pot when fully closed.	<code>KJO_Valve.h</code>
<code>OX_VALVE_BALL_DIAMETER</code>	0.611 in	Ball outer diameter used in the Ox valve's angle-based flow-position geometry calculation.	<code>KJO_Valve.h</code>
<code>OX_VALVE_BORE_DIAMETER</code>	0.357 in	Bore diameter used in the Ox valve's flow-position geometry calculation.	<code>KJO_Valve.h</code>

OX_VALVE_THROAT_DIAMETER	0.373 in	Throat inner diameter used in the Ox valve's flow-position geometry calculation.	KJO_Valve.h
FUEL_VALVE_PWM_OPEN	2200	12-bit PWM count for the Fuel valve's fully-open servo position.	KJO_Valve.h
FUEL_VALVE_PWM_CLOSE	3800	12-bit PWM count for the Fuel valve's fully-closed servo position.	KJO_Valve.h
FUEL_VALVE_POS_OPEN	1108	ADC count from the Fuel valve position pot when fully open.	KJO_Valve.h
FUEL_VALVE_POS_CLOSED	2786	ADC count from the Fuel valve position pot when fully closed.	KJO_Valve.h
FUEL_VALVE_BALL_DIAMETER	0.611 in	Ball outer diameter used in the Fuel valve's flow-position geometry calculation.	KJO_Valve.h
FUEL_VALVE_BORE_DIAMETER	0.357 in	Bore diameter used in the Fuel valve's flow-position geometry calculation.	KJO_Valve.h
FUEL_VALVE_THROAT_DIAMETER	0.373 in	Throat inner diameter used in the Fuel valve's flow-position geometry calculation.	KJO_Valve.h
VALVE_MOVE_TIMEOUT_MS (alias MOVE_TIMEOUT)	2500 ms	Maximum time allowed for a commanded valve move to complete before the watchdog gives up; shared with GSEMU since both use the identical servo/gear actuator.	KJO_Valve_Timing.h (shared)
VALVE_POSITION_AVERAGING_COUNT	4	Number of ADC reads averaged for each valve position sample.	KJO_Valve.h
VALVE_MOVING_THRESHOLD	2 counts	Minimum encoder delta between samples required to consider the valve still moving.	KJO_Valve.h
VALVE_POSITION_DEAD_BAND	15 counts	Position tolerance used to decide whether a valve has reached its open/closed target.	KJO_Valve.h
MOVE_DETECT_WAIT	30 ms	Minimum interval between successive encoder samples inside <code>isMoving()</code> .	KJO_Valve.h
MOVE_AFTER_START_WAIT	60 ms	Blocking delay after issuing a servo PWM command, giving the servo time to begin responding before motion is sampled.	KJO_Valve.h
USE_MAPPING	true	Selects angle-geometry-based valve position mapping over simple linear PWM interpolation.	KJO_System_Config.h (shared)

Pressure Sensing & Calibration

Parameter	Value	Description	Source File
PRESSURE_AVERAGING_COUNT	4	Number of ADS1015 reads averaged per pressure measurement.	KJO_Pressure.h
CHAMBER_PRESSURE_SLOPE	0.137533	Bench-calibrated linear-fit slope (PSI per ADC count) for the chamber pressure transducer.	KJO_Pressure.h
CHAMBER_PRESSURE_INTERCEPT	-25.012515	Bench-calibrated linear-fit intercept (PSI at 0 counts) for the chamber pressure transducer.	KJO_Pressure.h
TANK_PRESSURE_SLOPE	0.688427	Bench-calibrated linear-fit slope (PSI per ADC count) for the Ox tank pressure transducer.	KJO_Pressure.h
TANK_PRESSURE_INTERCEPT	-124.131326	Bench-calibrated linear-fit intercept (PSI at 0 counts) for the Ox tank pressure transducer.	KJO_Pressure.h
TANK_MAX_PSI	1000 PSI	Full-scale rating of the tank pressure transducer, used for header/documentation purposes.	KJO_Pressure.h
CHAMBER_MAX_PSI	200 PSI	Full-scale rating of the chamber pressure transducer, used for header/documentation purposes.	KJO_Pressure.h

PRESSURE_WARNING_THRESHOLD	15 PSI	Tank pressure above which the red indicator and pressure-alarm buzzer activate, and below which recording auto-stops.	KJO_Pressure.h
PRESSURE_CHECK_INTERVAL_MS	250 ms	Poll interval for the unconditional tank-pressure safety check (<code>Check_Pressure_Warning()</code>).	KJO_Pressure.h
ABORT_VENTED_THRESHOLD_PSI	5 PSI	Tank gauge pressure below which an in-progress Abort's vent-wait step considers the tank vented and latches.	KJO_Pressure.h
MAX_OX_TANK_PRESSURE	950 PSI	Sustained Ox-tank gauge pressure threshold that triggers an automatic Abort.	KJO_Pressure.h
OX_TANK_PRESSURE_TIME_MS	1000 ms	How long the Ox tank must remain at/above <code>MAX_OX_TANK_PRESSURE</code> before the automatic abort fires.	KJO_Pressure.h
AD_BASE_SCALE	0.00199144777 V/count	ADS1015 volts-per-count scale factor at PGA gain 1, used for battery-voltage conversion.	KJO_Analog.h
LIPO_SCALE	2.79111	Resistor-divider scale factor converting the ADS1015 LiPo channel reading to actual battery voltage.	KJO_Analog.h
AIRSTART_THRESHOLD (alias of LCO_SENSE_THRESHOLD)	251 counts	ADC-count threshold above which the Airstart/LCO sense channel is considered triggered (~ 0.5 V post-divider).	KJO_LCO_Sense.h (shared)

Thermocouple Sensing

Parameter	Value	Description	Source File
TEMPERATURE_TC_AVERAGING_COUNT	4	Number of thermocouple readings averaged per measurement (declared but not yet wired into a read path).	KJO_Thermocouple.h

Engine Start Sequence Timing

Parameter	Value	Description	Source File
IGNITER_FIRE_DURATION_MS	2000 ms	Duration the igniter relay stays energized to reliably light the solid starter motor.	main.cpp
START_DWELL_MS	2000 ms	Fixed pause after confirmed umbilical separation before propellant flow begins.	main.cpp
FUEL_FLOW_DELAY_MS	500 ms	Placeholder transit-time allowance after Fuel valve confirms open, before proceeding (pending cold-flow calibration).	main.cpp
OX_FLOW_DELAY_MS	200 ms	Placeholder transit-time allowance after Ox valve confirms open, before igniter fires (pending cold-flow calibration).	main.cpp
QR_RELEASE_TIMEOUT_MS	3000 ms	Maximum time to wait for physical umbilical separation after commanding QR release, before aborting the start sequence.	main.cpp
ABORT_VALVE_CONFIRM_TIMEOUT_MS	MOVE_TIMEOUT + 100 ms (2600 ms)	Watchdog for each Abort-sequence valve-close/open confirmation step.	main.cpp
EMU_GSEMU_TIMEOUT_MS	200 ms	CAN round-trip timeout for ordinary EMU-to-GSEMU relay requests (no chaining through GSEMU).	main.cpp
EMU_LCMU_TIMEOUT_MS	200 ms	CAN round-trip timeout for ordinary EMU-to-LCMU relay requests.	main.cpp

EMU_GSEMU_FILL_ VALVE_MOVE_TIMEOUT_ MS	MOVE_TIMEOUT + 300 ms (2800 ms)	CAN timeout for GSEMU Fill-valve open/close requests, sized to cover GSEMU's own valve-move watchdog plus round-trip margin.	main.cpp
Motor_Ox_Fill_ Target_Lbm() values (L/M/N/O)	7.026 / 14.053 / 28.106 / 58.219 lbm	Per-motor-designation Ox fill target weight EMU's fill session compares polled LCMU readings against.	main.cpp
SELECTED_MOTOR	Motor_ Designation::M	Selects which motor's Ox fill target is active for the current test article; changed per test.	main.cpp

Abort / Indicator / Buzzer Timing

Parameter	Value	Description	Source File
LINK_LOS_INTERVAL_ MS	15000 ms	Time without a received radio packet before the RCU link is declared lost.	main.cpp
ALIVE_BEEP_ INTERVAL_MS	10000 ms	Period of the idle "I'm alive" buzzer beep.	main.cpp
FILL_BEEP_INTERVAL_ MS	2000 ms	Period of the buzzer beep while a fill session is active (filling or paused), replacing the alive beep.	main.cpp
ABORT_LATCH_COLOR_ INTERVAL_MS	1000 ms	Red/yellow alternation period (per color) for the indicator once ABORT_LATCHED.	main.cpp
BOOT_RETRY_BACKOFF_ MS	3000 ms	Delay before retrying a timed-out boot-time CAN presence ping (GSEMU/LCMU).	main.cpp
PRESENCE_PING_ TIMEOUT_MS	200 ms	CAN round-trip timeout for the runtime GSEMU/LCMU presence ping.	main.cpp
PRESENCE_PING_ INTERVAL_MS	2000 ms	Interval between successive full GSEMU+LCMU presence-ping cycles.	main.cpp
FILL_POLL_INTERVAL_ MS	200 ms	Poll interval for the background LCMU weight query during an active fill.	main.cpp

Continuous Sensor Recording / Burn Data Capture

Parameter	Value	Description	Source File
PRESSURE_RECORDING_ INTERVAL_MS	250 ms	Sample period for continuous pres- sure/temperature recording while active.	main.cpp
PRESSURE_AMBIENT_ COUNT	4	Consecutive below-threshold pressure sam- ples required to confirm ambient and auto- stop recording.	main.cpp
BURN_TRIGGER_DELTA_ PSI	5.0 PSI	Chamber gauge pressure rise above tared ambient that starts the burn-trigger sustain timer.	main.cpp
BURN_TRIGGER_ SUSTAIN_MS	1000 ms	How long chamber pressure must stay above BURN_TRIGGER_DELTA_PSI before the burn recording is considered triggered.	main.cpp
BURN_SAMPLE_ INTERVAL_MS	100 ms (10 Hz)	Sample rate for burn-data capture.	main.cpp
BURN_PRE_TRIGGER_MS	10000 ms	Amount of pre-trigger history retained in the burn ring buffer.	main.cpp
BURN_POST_TRIGGER_ DURATION_MS	30000 ms	Fixed capture duration after the burn trigger fires.	main.cpp
BURN_NEVER_ TRIGGERED_TIMEOUT_ MS	60000 ms	Safety-net duration after arming; if the burn never triggers, recording completes anyway.	main.cpp

BURN_BUFFER_MAX_POST_SAMPLES	300 (30 s @ 10 Hz)	Fixed capacity of the post-trigger burn sample buffer.	KJO_Burn_Buffer.h
BURN_BUFFER_MAX_PRE_SAMPLES	100 (10 s @ 10 Hz)	Fixed capacity of the pre-trigger burn ring buffer.	KJO_Burn_Buffer.h
BURN_BUFFER_NUM_CHANNELS	4	Number of sensor channels stored per burn sample (tank PSI, chamber PSI, Temp1, Temp2).	KJO_Burn_Buffer.h

ADC Diagnostic Mode (bench-only, compiled out by default)

Parameter	Value	Description	Source File
ADS_DIAG_LSB_V	4.096/2048 V/count	ADS1015 GAIN_ONE LSB voltage used to back-calculate connector voltage during bench diagnostics.	main.cpp
ADS_DIAG_DIV_INV	4.5/3.3	Voltage-divider inverse ratio for the chamber channel diagnostic readback.	main.cpp
ADS_DIAG_DIV_INV_TANK	(4.5 × 3.290)/(3.3 × 3.267) ≈ 1.37324	Bench-trimmed voltage-divider inverse ratio for the tank channel diagnostic readback.	main.cpp
ADC_DIAG_INTERVAL_MS	2000 ms	How often the ADC diagnostic reading is printed to the OLED.	main.cpp

Radio Communication

Parameter	Value	Description	Source File
RCS_CENTER_FREQUENCY	917.117 MHz	LoRa center frequency; must match between EMU and RCU radios to communicate.	KJO_Radio.h (shared)
TRANSMIT_POWER	5 dBm	LoRa transmit power setting (valid range 5–23 dBm via PA_BOOST).	KJO_Radio.h (shared)
LINK_MARGINAL_THRESHOLD	−90 dBm	RSSI below which the link-quality indicator shows MARGINAL rather than OK.	KJO_Radio.h (shared)
MAX_PAYLOAD_LENGTH	64 bytes	Maximum LoRa packet payload size EMU/RCU buffers allocate (headroom under RadioHead's 251-byte ceiling).	KJO_Command_Defs.h (shared)
MAX_COMMAND_CODE	39	Highest valid LoRa command code, used to range-check incoming packets.	KJO_Command_Defs.h (shared)

CAN Bus Communication Timing

Parameter	Value	Description	Source File
CAN_BAUDRATE	1,000,000 bps	CAN bus bit rate shared by EMU/GSEMU/LCMU; raised from 250 kbps, safely within spec for the installation's short (16 ft max) bus runs.	KJO_CAN_Command_Defs.h (shared)
CAN_MAX_COMMAND_CODE	23	Highest valid CAN command code, used to bounds-check inbound frames before name-table lookup.	KJO_CAN_Command_Defs.h (shared)

SD Logging

Parameter	Value	Description	Source File

LOG_FILE_NAME_BASE	"Log_"	Base filename prefix for sequentially-numbered SD log files (e.g. Log_0.txt).	KJO_Logging.h
--------------------	--------	---	---------------

11 Version History

Version	Date	Notes
0.1	March 2026	Initial release.
0.2	March 2026	Add umbilical QR release subsystem (QR_Master, QR_RELEASE command, AUX_IO_3 protocol).
0.3	March 2026	Level-based QR protocol (release=LOW, hold=HIGH); add RELEASE_STATE_EIO, LAUNCH_ENABLE_EIO, REMOTE_START_EIO; Button A reassigned to hold-to-open Dump valve; QR_Master API updated.
0.4	March 2026	Full command dispatch table; ENGINE_STARTING state added to state machine; 7-step engine start sequence documented; new §7 Buzzer and Visual Indicator: 5-beep startup, NeoPixel state machine (5 states/priorities), continuous pressure-alarm buzzer, 10 s alive beep, alarm silence on start/abort.
0.5	March 2026	Valve servo calibration: added §3 servo geometry subsection (gear ratio 3.75:1, 1100 Hz PCA9685, ≈ 1690 theoretical counts for 90°) and empirical per-valve calibration table (all four valves re-calibrated on bench). Pressure transducer fix: corrected ADS1015 channel assignments (chamber = ch1/200 PSI, tank = ch2/1000 PSI), corrected sensor table (temperature sensors use onboard ADC A0/A5, not ADS1015), added §7 signal chain subsection documenting voltage divider and calibration endpoint counts (184/1657).
0.6	March 2026	SD log files moved to SD card root (Log_N.txt); subdirectory path constants (LOG_FILE_FOLDER, LOG_FILE_DIR) removed from KJO_Logging.h and KJO_EMU_Display.
0.7	July 2026	Dump valve physically removed from the vehicle; pressurant is now dumped through the combustion chamber via the Ox valve. DUMP/STOP_DUMP retained but now target the Ox valve; Execute_Abort() closes Fill/Fuel and opens Ox (instead of closing all valves); Button A reassigned to hold-to-open the Ox valve; dump_pct removed from Heartbeat_Response_Packet.
0.8	September 2026	Add new §10 Parameters & Flags reference section: tunable timing, threshold, and calibration constants across the valve, pressure, engine-start, indicator, burn-recording, radio, and CAN subsystems, with source-file attribution for each.
0.9	September 2026	Corrected stale wired-GPIO descriptions left over from the CAN redesign: removed §3.5 Master_Valve Class (the class no longer exists); rewrote §4 as “Fill Valve CAN Protocol” describing the actual CAN_OPEN_FILL_VALVE/CAN_CLOSE_FILL_VALVE mechanism; rewrote §6 Umbilical Quick-Release to describe QR_Master as a sense-only reader and CAN_QR_RELEASE as the real release path; corrected 5 of 6 AUX_IO pin-map rows (only BUTTON_A_EIO’s row and the umbilical/link-sense functions survive; Fill valve CMD/STATUS and the old QR/launch/remote-start lines are retired); added new §3.5 Local Button Functions documenting all three front-panel buttons (only Button A is still functional); corrected two stale Command Table rows (BEGIN_FILL, QR_RELEASE).